N Queen

```
board = [-1] * 8

def safe(row, col):
    for i in range(row):

        # Same column
        if board[i] == col:
            return False

        # Same diagonal
        if abs(board[i] - col) == abs(i - row):
            return False

    return True

def solve(row):

    if row == 8:
        return True

    for col in range(8):

        if safe(row, col):

            board[row] = col

            if solve(row + 1):
                return True

            board[row] = -1

    return False

if solve(0):
    print(board)
```

Cannibals

```
from collections import deque

def is_valid(m, c):
    if m < 0 or m > 3 or c < 0 or c > 3:
        return False

    if m > 0 and m < c:
        return False
```

```
    mr = 3 - m
    cr = 3 - c

    if mr > 0 and mr < cr:
        return False

    return True


def bfs():
    start = (3, 3, 1)   # (Missionaries, Cannibals, Boat)
    goal = (0, 0, 0)

    moves = [
        (1, 0),  # 1 Missionary
        (2, 0),  # 2 Missionaries
        (0, 1),  # 1 Cannibal
        (0, 2),  # 2 Cannibals
        (1, 1)   # 1 Missionary + 1 Cannibal
    ]

    queue = deque([start])
    visited = {start}
    parent = {}

    while queue:
        m, c, boat = queue.popleft()

        if (m, c, boat) == goal:
            path = []
            state = goal

            while state != start:
                path.append(state)
                state = parent[state]

            path.append(start)
            path.reverse()

            print("Solution Path:")
            for step, s in enumerate(path):
                print(f"Step {step}: {s}")
            return

        for dm, dc in moves:

            if boat == 1:  # Boat on left side
                nm, nc, nb = m - dm, c - dc, 0
            else:          # Boat on right side
                nm, nc, nb = m + dm, c + dc, 1
```

```
            new_state = (nm, nc, nb)

            if is_valid(nm, nc) and new_state not in visited:
                visited.add(new_state)
                parent[new_state] = (m, c, boat)
                queue.append(new_state)

    print("No Solution Found")


bfs()
```

JUG

```
from collections import deque

CAP_A = 3
CAP_B = 4
TARGET = 2

def bfs():
    start = (0, 0)
    queue = deque([start])
    parent = {start: None}

    def neighbors(a, b):
        res = []

        # Fill Jug A
        res.append((CAP_A, b))

        # Fill Jug B
        res.append((a, CAP_B))

        # Empty Jug A
        res.append((0, b))

        # Empty Jug B
        res.append((a, 0))

        # Pour A -> B
        pour = min(a, CAP_B - b)
        res.append((a - pour, b + pour))

        # Pour B -> A
        pour = min(b, CAP_A - a)
        res.append((a + pour, b - pour))
```

```
return res

    while queue:
        a, b = queue.popleft()

        if a == TARGET or b == TARGET:
            path = []
            cur = (a, b)

            while cur is not None:
                path.append(cur)
                cur = parent[cur]

            path.reverse()

            print("Solution Path (A, B):")
            for i, (x, y) in enumerate(path):
                print(f"Step {i}: A = {x}L, B = {y}L")
            return

        for na, nb in neighbors(a, b):
            if (na, nb) not in parent:
                parent[(na, nb)] = (a, b)
                queue.append((na, nb))

    print("No solution found.")

if __name__ == "__main__":
    bfs()
```

l Hill *climbing*

```
import random

dist = [
    [0, 400, 500, 300],  # Node 0 (A)
    [400, 0, 300, 500],  # Node 1 (B)
    [500, 300, 0, 400],  # Node 2 (C)
    [300, 500, 400, 0]   # Node 3 (D)
]

N = len(dist)

def tour_cost(tour):
    total = 0
    for k in range(len(tour)):
        total += dist[tour[k]][tour[(k + 1) % len(tour)]]
    return total
```

```
def reverse_segment(tour, i, j):
    return tour[:i] + tour[i:j + 1][::-1] + tour[j + 1:]

def cheapest_hill_climbing(start_tour):
    tour = start_tour[:]
    current_cost = tour_cost(tour)
    step = 0

    print("=" * 60)
    print("INITIAL TOUR :", " -> ".join(str(c) for c in tour))
    print("INITIAL COST :", current_cost)
    print("=" * 60)

    while True:
        best_tour = None
        best_gain = 0

        for i in range(1, N - 1):
            for j in range(i + 1, N):
                new_tour = reverse_segment(tour, i, j)
                new_cost = tour_cost(new_tour)

                gain = current_cost - new_cost

                if gain > best_gain:
                    best_gain = gain
                    best_tour = new_tour

        if best_tour is None or best_gain <= 0:
            break

        tour = best_tour
        current_cost -= best_gain
        step += 1

        print(
            "Step %2d : cost improved by %6d | cost now = %6d | tour = %s"
            % (
                step,
                best_gain,
                current_cost,
                " -> ".join(str(c) for c in tour)
            )
        )

    print("=" * 60)
    print("FINAL TOUR :", " -> ".join(str(c) for c in tour))
    print("FINAL COST :", current_cost)
    print("LOCAL OPTIMUM after %d improvement(s)" % step)
```

```
    return tour, current_cost

if __name__ == "__main__":
    random.seed(42)

    start_tour = list(range(N))
    random.shuffle(start_tour)

    best_tour, best_cost = cheapest_hill_climbing(start_tour)
```

Tic **tac** Toe

```
import math

def check_win(b):
    wins = [
        (0, 1, 2), (3, 4, 5), (6, 7, 8),
        (0, 3, 6), (1, 4, 7), (2, 5, 8),
        (0, 4, 8), (2, 4, 6)
    ]

    for i, j, k in wins:
        if b[i] == b[j] == b[k] and b[i] != ' ':
            return b[i]

    return None


def alpha_beta(b, is_max, alpha, beta):
    winner = check_win(b)

    if winner == 'X':
        return 10

    if winner == 'O':
        return -10

    if ' ' not in b:
        return 0

    if is_max:
        best = -math.inf

        for i in range(9):
            if b[i] == ' ':
                b[i] = 'X'

                val = alpha_beta(b, False, alpha, beta)
```

```
            b[i] = ' '

            best = max(best, val)
            alpha = max(alpha, best)

            if beta <= alpha:
                break

        return best

    else:
        best = math.inf

        for i in range(9):
            if b[i] == ' ':
                b[i] = 'O'

                val = alpha_beta(b, True, alpha, beta)

                b[i] = ' '

                best = min(best, val)
                beta = min(beta, best)

                if beta <= alpha:
                    break

        return best


board = [
    'O', ' ', ' ',
    ' ', 'X', ' ',
    ' ', ' ', 'O'
]

best_score = -math.inf
best_move = -1

for i in range(9):
    if board[i] == ' ':
        board[i] = 'X'

        score = alpha_beta(board, False, -math.inf, math.inf)

        board[i] = ' '

        if score > best_score:
            best_score = score
```

```
            best_move = i

print("Current Board:")
print(board[0:3])
print(board[3:6])
print(board[6:9])

print("Optimal AI ('X') Move Index:", best_move)
```